

IA Aplicada com LLMs

Aula 05 — Padrões de arquitetura de agentes

Onde paramos

A Aula 04 apresentou uma dúzia de casos de indústria sob a pergunta "*que padrão é este?*". A resposta possível era um vocabulário **de nomes**.

E a Aula 03 encerrou com uma lista de pendências:

estado e memória · múltiplas ferramentas · outros padrões de arquitetura · MCP · confiabilidade · context engineering dinâmica

Hoje os nomes viram código, e quatro pendências são encerradas.

O laço entregue pela Aula 03

```
for passo in range(max_passos):  
    r = client.chat.completions.create(messages=mensagens, tools=DECLARACOES)  
    if not r.choices[0].message.tool_calls:  
        return r.choices[0].message.content  
    ...
```

Três limitações o impedem de operar sem supervisão:

1. **Não sabe parar** — `max_passos` é teto, não orçamento → nota 03
2. **Não sabe falhar** — uma exceção derruba o processo → nota 03
3. **Não sabe esquecer** — o histórico apenas cresce → nota 04

E uma questão anterior a todas: a tarefa exigia um laço?

Este deck trata dessa questão.

Com um case já escolhido, a pergunta pertinente em cada padrão não é de compreensão, e sim de aplicação: **é este o padrão que o case exige?**

Roteiro: a linha divisória · autópsia do exercício 03 ·
cinco padrões · a tabela de decisão · composição

Parte 1

A linha divisória

De definição a critério operacional

Workflow: o caminho está no código.

Agente: o caminho é decidido em execução pelo modelo.

Definição adequada para exposição, insuficiente para decisão de projeto.

O critério é se o fluxograma pode ser desenhado antes da execução.

Sim → escrever o fluxograma em Python.

Não → há motivo para autonomia.

Três perguntas de apoio

	Sim	Não
O nº de passos é conhecido?	workflow	agente
A ordem depende de dados de execução?	agente	workflow
O erro é caro ou irreversível?	reduzir a autonomia	tolerável

A terceira não trata da natureza da tarefa — trata da **consequência do erro**.

Tarefa que admitiria um agente, mas cujo erro emite dinheiro, é tarefa em que se aceita um sistema pior em troca de previsibilidade.

O preço de subir no espectro

Nível	Chamadas	Previsível	Depurável
Prompt único	1	sim	sim
Chain	N fixo	sim	sim
<i>Router</i>	1 + rota	sim	sim
Agente	?	não	apenas com trace
Multiagente	??	não	muito difícil

A última coluna é a mais onerosa: num agente a trajetória difere a cada execução e, sem log, resta uma saída incorreta sem indício de causa.

O workflow disfarçado de agente mais próximo

...foi produzido pela turma na aula anterior.

Etapa	Classificação
1. Coleta	workflow — a condição de parada é um <code>if</code>
2. Consulta	workflow — sempre, e nessa posição
3. Raciocínio	decisão — de uma, não de uma sequência
4. Ação	workflow condicional
5. Consulta final	workflow — sempre por último

Quatro das cinco etapas estavam no código

A sequência constava **numerada no enunciado**.

Isso não constitui erro: o objetivo daquele exercício era o **mecanismo**.

Em produção, porém, a versão em workflow seria superior em todos os eixos: menos chamadas, sem risco de omitir a consulta final ou de abrir dois chamados, testável etapa por etapa. Perderia apenas o atendimento fora do roteiro — e é nesse ponto que a decisão se coloca.

Quando o enunciado vem numerado, o código também deveria vir.

Parte 2

Os cinco padrões

Taxonomia de ANTHROPIC (2024)

O que é um padrão, aqui

Não é um diagrama nem uma biblioteca. É a resposta a **três perguntas**:

	A pergunta
Quem decide o fluxo	o seu código, ou o modelo em tempo de execução?
Quantas chamadas custa	o número é conhecido antes de rodar?
Quando <i>não</i> usar	qual é a condição em que ele é a escolha errada?

A terceira é a que a literatura de mercado omite.

Padrão sem condição de não uso é propaganda, e não engenharia.

1. Sequencial (prompt chaining) com portão

```
entrada —> [ LLM 1 ] —> PORTÃO —> [ LLM 2 ] —> saída
           extrai                |
                               falhou —> para aqui
                               redige
```

```
extraido = extrair(mensagem)          # etapa 1: o que o CLIENTE disse
pedido   = portao(extraido)           # o PORTÃO: código, não modelo
                                             # devolve o que o SISTEMA sabe
return redigir(mensagem, pedido)      # etapa 2
```

Regra expressável em Python é implementada em Python: mais barata, e não alucina.

Não usar: etapas interdependentes · nº de etapas dependente da entrada

1. O portão: duas naturezas de validação

Natureza	A pergunta	Exemplo
Forma	o campo tem o formato certo?	o número tem 5 dígitos?
Fato	o que o cliente disse é verdade ?	esse pedido existe ? a data já passou?

O modelo simula a primeira. **A segunda ele não tem como verificar** — ele não conhece a base de pedidos.

O portão não melhora a resposta: impede que uma errada seja produzida.

Parar no portão custa **uma** chamada. Não parar custa duas, e uma resposta errada entregue ao cliente.

1. O que o laboratório mostra

```
"Meu pedido 99999 está parado há dias, isso é inaceitável!"
```

```
[etapa 1]  pedido=99999    tipo=atraso
```

```
[portão ]  BARRADA – o pedido 99999 não existe na base
```

```
[etapa 2]  não executada
```

```
[SEM portão, o que sairia]
```

```
"Verificamos o pedido 99999 e ele consta como em trânsito..."
```

Impecavelmente redigida. E sobre um pedido que **não existe**.

O que a etapa 2 recebe quando passa: **os dados do sistema**, não os do cliente. O que ele escreveu serviu para localizar o pedido, e só.

2. Router (Roteador) — o modelo classifica, o código despacha

```
entrada —> [ classificador ] —> rota A: código puro (sem LLM)
                        |
                        | —> rota B: outro prompt / outro modelo
                        |
                        | —> rota C: humano
                        |
                        | —> nenhuma —> fila de revisão
```

```
r = classificar(item, schema_com_enum(ROTAS))    # 1 chamada, barata
if r["rota"] == "dentro_da_politica":
    return aprovar_por_regra(item)                # ZERO chamadas de LLM
if r["rota"] == "ambiguo": return agente_analisa(item)
if r["rota"] == "acima_da_alcada": return encaminhar_humano(item)
return fila_de_revisao(item)                      # "nenhuma": não inferir
```

A rota **nenhuma** é obrigatória — sem escape, o modelo elege opção inaplicável com alta confiança. A de maior valor é a que não chama o modelo.

Não usar: rotas quase idênticas · classificação trivial em `if`

O classificador não precisa ser um LLM

Forma	Como decide	Preço
Regra	palavra-chave, regex, metadado	zero chamadas — não alucina, não generaliza
Embedding	vetor da entrada × exemplos de cada categoria	uma chamada barata, sem gerar texto
Semântico	modelo leve devolve <code>{"rota": "..."} </code>	uma chamada de geração

As três se combinam, e o laboratório de hoje usa a primeira e a terceira.

A do meio é a **Aula 06** — aparece aqui nomeada, e não implementada.

Por que ele é fundamental

Eficiência	entrada simples vai para o modelo pequeno, ou para código sem modelo nenhum
Especialização	com vários agentes, o <i>Router</i> decide quem assume — embrião da Aula 12
Ferramenta	decide se precisa de banco, de busca, ou de nenhum dos dois — Aula 10

O Router é onde a arquitetura fica barata.

As quatro entradas do lote de hoje

"Qual o status do pedido 48219?"

-> REGRA, em código. Zero chamadas de LLM

"A caixa do 55870 chegou rasgada"

-> MODELO, rota `reclamacao`

"Consta que o 77310 foi entregue mas eu não recebi nada"

-> MODELO – divergência: é o caso que justifica o AGENTE

"meu pedido não chegou"

-> falta o número -> `nenhuma` -> fila de revisão

A **primeira** é a maioria do lote. A **última** revela se há rota de escape.

3. Paralelização — não é concorrente do Router

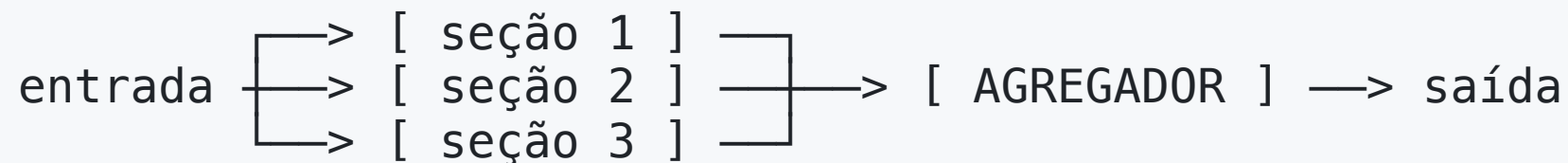
	O que faz com os caminhos
<i>Router</i>	escolhe um e descarta os outros
Paralelização	executa vários ao mesmo tempo e combina

É **padrão de execução**, não arquitetura — roda *dentro* de uma, inclusive de uma que já tem Router na frente.

E o ganho é TEMPO, não dinheiro. Três chamadas em paralelo custam os mesmos tokens que três em sequência.

O que cai é o tempo de parede: o da mais lenta, não a soma.

3a. Sectioning — divisão e conquista



os 5 pedidos em aberto

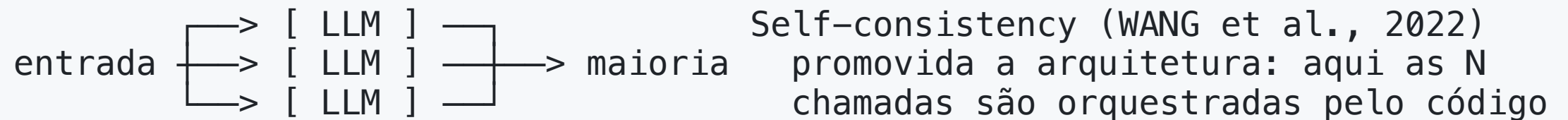
- "atrasos" quais passaram da previsão, e há quantos dias
- "carteira" quantos pedidos por transportadora
- "risco" quais vencem nos próximos 5 dias

AGREGADOR -> "3 em trânsito, 2 vencidos. A RápidoLog concentra o atraso."

As três **não se consultam** — e é esse teste que autoriza o padrão.

O agregador é um componente, não um `"\n".join()`.

3b. Voting — a mesma tarefa N vezes



Orquestrar em código permite variar **temperatura e modelo** entre as N.

O parâmetro `n` do provedor não permite: lá é o mesmo prompt, mesmo modelo.

3b. Voting — o que fazer com 2 de 3

"Consta que o 77310 foi entregue mas eu não recebi nada"

temp 0.0	-> reclamacao	"cliente afirma não ter recebido"
temp 0.7	-> reclamacao	"divergência com o registro"
modelo B	-> consulta_status	"o cliente quer saber onde está"

maioria: reclamacao, 2/3 -> confiança 0,67

Aceitar a maioria, se o erro é barato.

Tratar como empate e escalar, se não é — auditoria, parecer, diagnóstico.

Ali o consenso não é desempate: é **evidência de que a resposta é segura**.

3b. Voting — a divergência é informação

```
vereditos = [classificar(item) for _ in range(n)]  
rota, votos = Counter(v["rota"] for v in vereditos).most_common(1)[0]  
if votos < n // 2 + 1:  
    return {"rota": "nenhuma", "motivo": "sem maioria"}
```

Três vereditos distintos **não** indicam que se deva eleger um deles.

Indicam que o caso é difícil — e, provavelmente, que cabe encaminhamento a humano. É o sinal de incerteza de menor custo disponível.

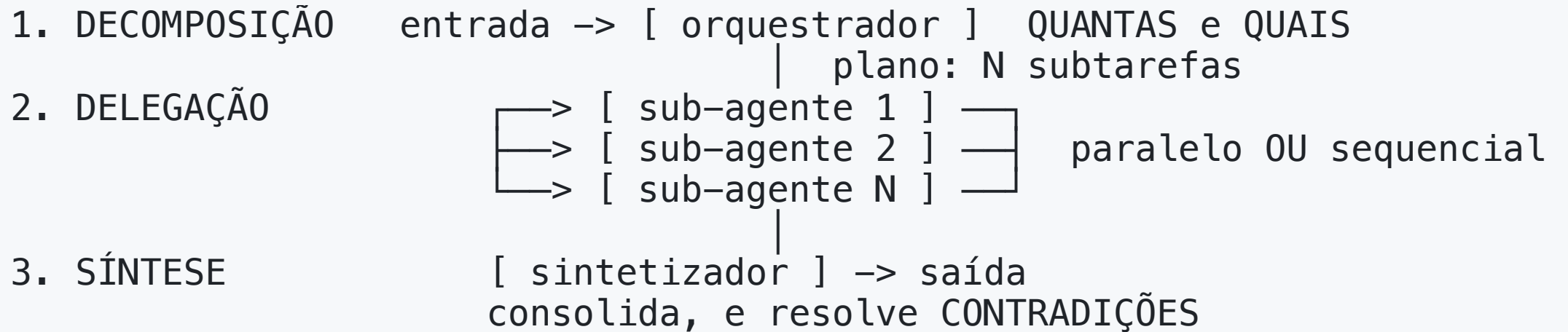
Não usar: *sectioning* com partes dependentes · *voting* em tarefa aberta ("redigir um resumo") — não há maioria entre três textos distintos

4. Orquestrador — Router e paralelização, com a decisão no modelo

	Quem decide	Quantos executam
<i>Router</i>	o modelo, entre rotas escritas pela aplicação	um
Paralelização	ninguém — está no código	N fixo
Orquestrador	o modelo, em execução	N desconhecido

O modelo decide quantas vezes ele mesmo será chamado, e é aí que a conta deixa de fechar antes de rodar.

4. As três fases



A fase 3 é a subestimada: o sintetizador **não concatena**. Ele recebe N respostas que podem se contradizer, e decide qual vale.

4. Paralelo ou sequencial — o critério é dependência

Se as subtarefas	Execução	Porque
são independentes	paralela	o tempo cai para o da mais lenta
uma consome a saída da outra	sequencial	não há como antecipar a segunda

Não é decisão de desempenho. É de **dependência** — a mesma pergunta do *sectioning*, agora sobre subtarefas que a aplicação não escreveu.

4. Por que sub-agentes, e não um agente só

O ganho é de **CONTEXTO**, não de organização.

Cada sub-agente recebe um *system prompt* curto e **só as ferramentas da sua especialidade**. Um agente único carregaria todas as declarações e todo o histórico em **toda volta** — e o consumo cresce com o quadrado dos passos.

E o preço: o que o sub-agente não incluiu, o orquestrador nunca saberá.

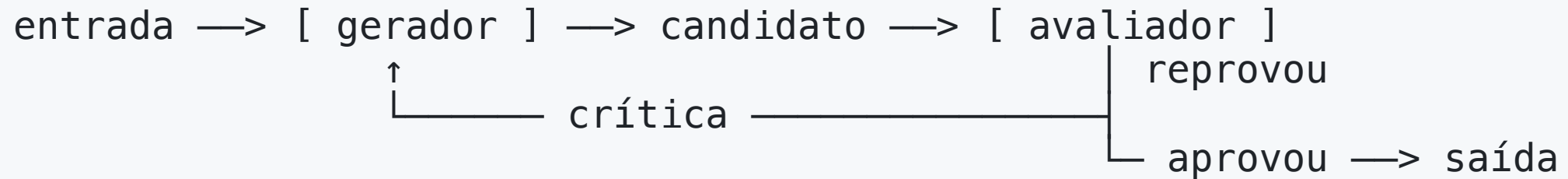
A perda não aparece como erro. Aparece como conclusão confiante sobre informação incompleta.

4. Orquestrador-trabalhador — o código

```
plano = chamar(PROMPT_ORQUESTRADOR, lote, schema=SCHEMA_PLANO)
if len(plano["subtarefas"]) > MAX_SUBTAREFAS:      # orçamento, já aqui
    raise PlanoGrandeDemais(len(plano["subtarefas"]))
resultados = [executar_subtarefa(s) for s in plano["subtarefas"]]
```

Distinção única e decisiva em relação ao *sectioning*: **as subtarefas só existem em execução**. Sem teto, trata-se de conta aberta assinada por um modelo. **Em caso de dúvida entre os dois, prefere-se sectioning.**

5. Avaliador-otimizador (SHINN et al., 2023)



```
for rodada in range(max_rodadas):  
    candidato = chamar(PROMPT_GERADOR, entrada, critica=critica)  
    aval = chamar(PROMPT_AVALIADOR, candidato, schema=SCHEMA_AVALIACAO)  
    if aval["aprovado"]: return candidato, "aprovado"  
    critica = aval["o_que_corrigir"]  
return candidato, "teto_de_rodadas"      # melhor esforço, sinalizado
```

5. Duas armadilhas sistemáticas

Sem critério escrito, o avaliador produz elogio.

```
{"nota": 8}                                # não acionável  
{"cita_regra": false, "o_que_corrigir": "..."} # acionável
```

Sem teto de rodadas, o par oscila. O avaliador exige A, o gerador entrega A e perde B, o avaliador exige B. O retorno precisa **declarar** a saída por teto — melhor esforço apresentado como aprovação é pior que reprovação explícita.

Não usar: quando o critério não é escrevível.

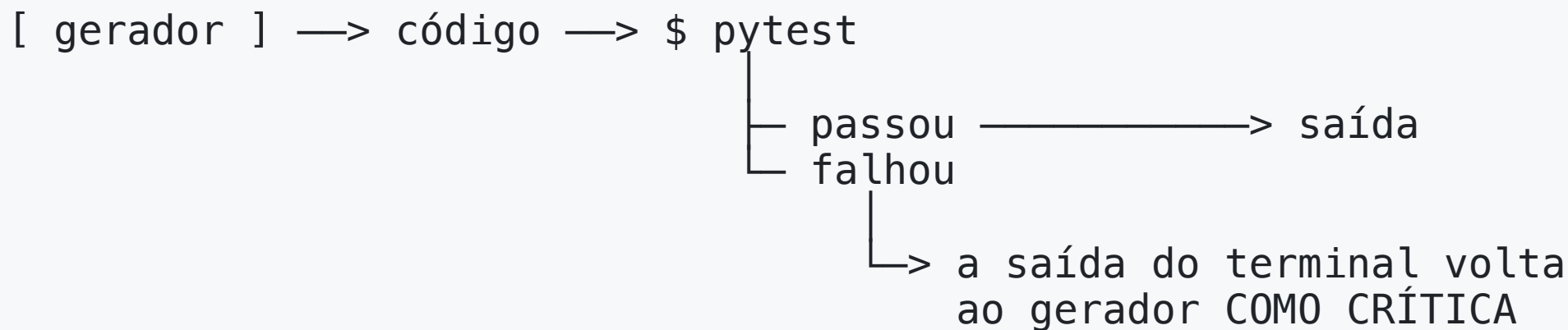
Nesse caso o avaliador adequado é uma pessoa.

5. Quem avalia — três modos, nesta ordem

Modo	Custo	Quando
Determinístico — teste, <i>linter</i> , schema	zero chamadas	o critério é executável
LLM como juiz — rubrica, item a item	1 por rodada	existe, mas não é executável
Humano no laço	trabalho humano	o erro é caro ou irreversível

Se dá para testar, o avaliador é o pytest. É a mesma tática zero do Router e da context engineering: o caminho de maior valor é o que não chama o modelo.

5. O agente de programação — o modo determinístico



Zero chamadas gastas na avaliação, e o *feedback* é **exato** em vez de aproximado: a linha e a exceção reais.

É a nota 05 §4 de novo — o **retorno de erro é prompt**.

E o terceiro modo já tem nome: é o `Termino.HUMAN0`, que a Aula 13 converte de conveniência em requisito.

5. O que o laço NÃO faz

Atribui-se a este padrão a redução de alucinação. **É impreciso, e caro.**

Um avaliador sem fonte externa não verifica fato: verifica se a resposta é **internamente consistente**. Um candidato que inventa um artigo inexistente e o cita de forma coerente em três rodadas é **aprovado** — e sai mais convincente do que entrou.

O que reduz alucinação é o critério verificável, e não o laço.

`cita_regra` só vale se alguém conferir a regra **contra o regulamento**.
É o defeito que a Aula 07 trata sob outro nome, e a razão de o RAG existir.

O eixo, antes da tabela

```
o CÓDIGO decide _____> o MODELO decide
prompt      sequencial Router sectioning orquestrador avaliador agente
único
└──────────┘ custo calculável └──────────┘ └── EXIGE TETO └──────────┘
```

A fronteira não está onde a intuição sugere. **No Router o modelo já decide** — mas entre rotas escritas *pela aplicação*, e o custo continua `1 + rota`.

A conta só quebra quando o modelo decide quantas vezes ele mesmo é chamado.

A tabela de decisão

Padrão	Quem decide o fluxo	Custo	Não usar quando
Prompt único	ninguém	1	exige validação por outra chamada
Sequencial + portão	o código	N fixo	etapas interdependentes
<i>Router</i>	o modelo, entre suas rotas	1 + rota	rotas quase idênticas
Sectioning	o código	1 por seção	partes dependentes
Voting	o código; a maioria	3–5	tarefa aberta
Orquestrador	o modelo, em	? — exige teto	as subtarefas são

Os quatro papéis, para levar na memória

Padrão	Papel	Analogia
<i>Router</i>	triagem e direcionamento	recepção
Paralelização	executar o independente ao mesmo tempo	equipe em paralelo
Orquestrador	planejar, dividir, coordenar, sintetizar	gerência de projeto
Avaliador	verificar e refinar por <i>feedback</i>	controle de qualidade

A analogia serve para lembrar o papel. **Ela não serve para escolher** — para isso existe a tabela anterior, que tem custo e condição de não uso.

Três perguntas que resolvem a maioria dos casos

1. O fluxograma existe?

Se sim, escreva-o em Python. Nenhum padrão desta parte é necessário.

2. É possível escrever `for parte in partes` sem chamar o modelo antes?

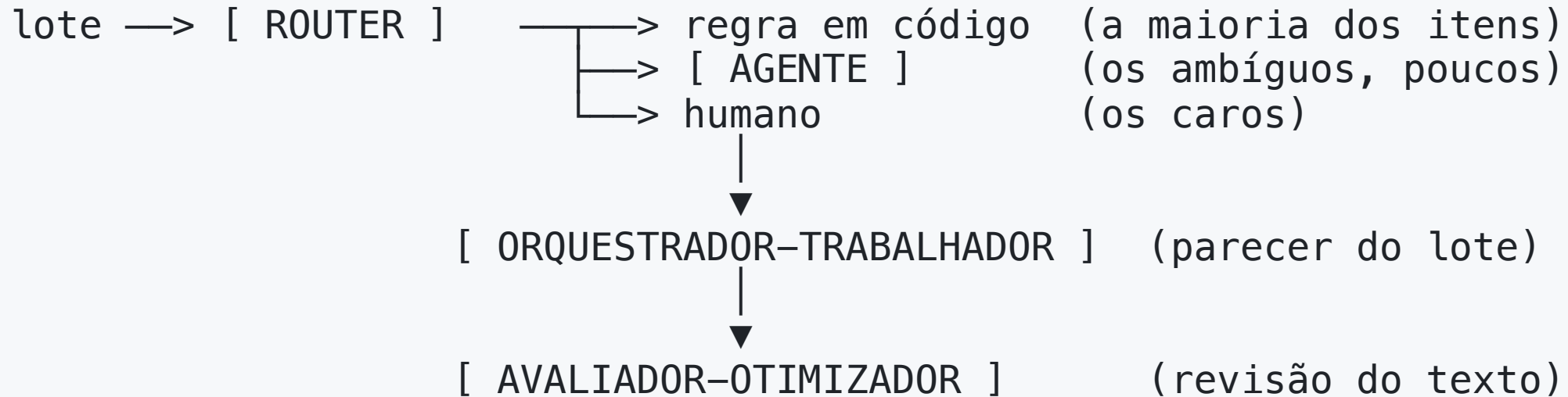
Se sim, é *sectioning*. Se não, é orquestrador — e ele precisa de **teto**.

3. O critério de qualidade é executável?

Se sim, o avaliador é um **teste**. Não um LLM, e não uma pessoa.

O erro mais caro não é escolher o padrão errado: é escolher um padrão quando nenhum era necessário.

Composição de padrões



Este é o desenho do exercício desta aula: a autonomia cara fica confinada ao caminho estreito em que é necessária, e o volume trafega por código determinístico.

Autonomia é recurso escasso. Gasta-se onde produz retorno.

O exemplo que encerra o deck

1.000 despesas, 15% em violação da política:

Arquitetura	Conta	Chamadas
Agente por despesa	$1.000 \times \sim 4$ passos	~ 4.000
Chain fixa	$1.000 + 150$	1.150
<i>Router</i> + regra em código	$0 + 300 + 150$	450

Redução de quase nove vezes — e os 70% resolvidos por regra são **mais** confiáveis, uma vez que regra não alucina. O ganho não decorre de prompt melhor nem de modelo melhor, e sim da decisão sobre **o que não enviar ao modelo**.